

MSA University

Project Title: *Network Intrusion Detection*

Course: CSE263 – Machine Learning and Pattern Recognition

S#	Student Name	Student ID
1	Abdelrahman Wagdy	241003
2	Gamila Hasan	242127
3	Mariam Mohamed Reyad	245171
4	Marina Ayman	246635
5	Mohamed Amr	243835

Date Submitted: 15/12/2025

1. Problem Description

Due to relying on computer networks extensively, the security of networks nowadays has become a critical and a huge concern to be accounted for and taken into consideration. Therefore, to be able to identify network attacks and unauthorized access, Intrusion Detection Systems (IDS) are designed to be utilized as a tool for identification of possible threats.

This project aims to utilize machine learning in detecting network intrusions. Through the utilization of historical labeled data from the NSL-KDD dataset, network connections will be classified to normal and various attack types. By using a user-friendly interface, the GUI, users will be able to get immediate classifications by uploading network logs in CSV format.

2. Data Preparation

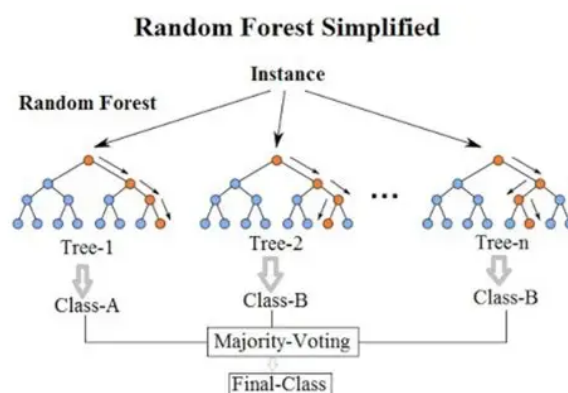
The project uses the *NSL-KDD* dataset, a standard benchmark for intrusion detection research.

Steps in data preparation:

1. **Loading Data:** The *KDDTrain+.txt* has 42 different features, including categorical and numerical features.
2. **Column Selection:** The feature *difficulty* is dropped as it does not contribute to the classification.
3. **Categorical Encoding:** Categorical features like: *protocol_type*, *flag* and *service* are encoded into numerical values using `LabelEncoder`.
4. **Feature Scaling:** Every numerical value is normalized using *StandardScaler()* to improve performance.

3. Methods and Models

The project utilizes the **Random Forest Model** for its ability to efficiently deal with high-dimensional data.



The **Random Forest** classifier is a machine learning algorithm that does a voting majority between many decision trees to make better predictions. Moreover, the **Random Forest** classifier works efficiently for both small and large datasets, handles missing values and shows feature importance.

Model Configuration:

- Number of trees = 150
- Maximum depth = 20
- Random state = 42

Workflow:

1. Load and preprocess the training data.
2. Train the Random Forest model on scaled features.
3. Use a **GUI** to allow CSV uploads for predictions.
4. Encode categorical features in the uploaded file using previously fitted encoders.
5. Normalize features, using the trained scaler.
6. Predict attack types and display results in the GUI.

The GUI enhances usability, providing instant feedback on detection results.

4. Results and Evaluation

- **Accuracy:** Measures how many out of all the predictions the model got *right*. In this project, accuracy visualizes the proportion of network connections that the model correctly labeled as normal or as a specific attack. Our model gave us:

Accuracy = 0.7224538679914834

which is a significantly high percentage that indicates a strong performance.

- **Precision, recall and F1 score:** *Precision* displays how many of the connections the model predicted as a specific attack were actually that attack, the higher the precision the fewer the false alarms. *Recall* displays how many of the connections of a specific attack the model correctly detected, the higher the recall, the fewer missed attacks. *F1 score* combines precision and recall into a single measure.

Sample of precision, recall, F1 score output:

	precision	recall	f1-score	support
apache2	0.00	0.00	0.00	737
back	0.86	0.97	0.91	359
buffer_overflow	0.50	0.05	0.09	20
ftp_write	0.00	0.00	0.00	3
guess_passwd	0.00	0.00	0.00	1231
httptunnel	0.00	0.00	0.00	133
imap	0.00	0.00	0.00	1
ipsweep	0.60	0.98	0.75	141
land	1.00	0.14	0.25	7
loadmodule	0.00	0.00	0.00	2
mailbomb	0.00	0.00	0.00	293
mscan	0.00	0.00	0.00	996
multihop	0.00	0.00	0.00	18
named	0.00	0.00	0.00	17
neptune	0.97	1.00	0.98	4657
nmap	0.89	1.00	0.94	73

Fig1: Sample output for precision, recall, F1 score

- **Confusion Matrix:** Shows the *actual labels* against the *predicted labels* for each class and it helps identify which attack types are often confused with others. Our confusion matrix shows that most classes are classified correctly. However, similar attacks were occasionally misclassified.

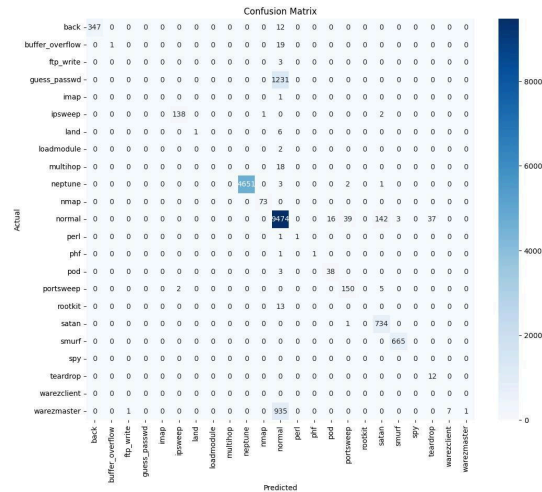


Fig2: Confusion Matrix Output

- **GUI Display:** Allows users to see how the model classifies each connection without having to analyze raw output.

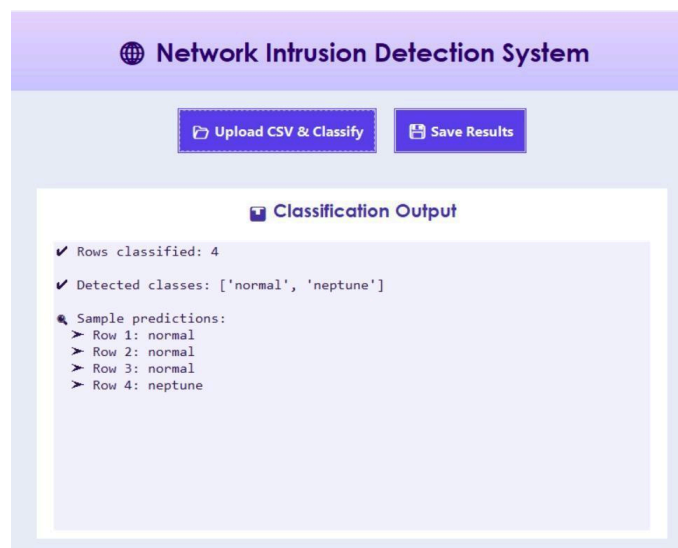


Fig3: GUI Display

5. Discussion

- **Model Strengths:** Random Forest is able to work well with many features and mixed data types and it reduces overfitting through combining multiple decision trees.
- **GUI Benefits:** GUI allows the model to be user-friendly as non-technical users will be able to interact with the IDS efficiently and classify network data easily.
- **Limitations and Future Work:** Although the model is accurate, it can sometimes confuse certain attacks if they have very similar network features so enhancements such as model optimization or real-time monitoring can help to further improve performance.

6. Appendix

A. Code Structure Overview:

Our implemented system is organized into clearly defined parts to ensure maintainability and readability. It begins with importing the required libraries needed for data processing, machine learning, visualization and GUI development. Then the *NSL-KDD* dataset is loaded and meaningful column names are assigned.

Moreover, the preprocessing section is for encoding categorical features to numerical ones and for normalization of numerical values. When all values are preprocessed, the **Random Forest** model gets trained with the prepared, preprocessed training data. Another section is for assessing the model's performance using the standard metrics. Lastly, the GUI section is for integrating the trained model in order to allow prediction, user interaction and result storage.

```
import pandas as pd
import tkinter as tk
from tkinter import FileDialog, messagebox, ttk
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier

#Training the model
train_path = "nsldata/train[1].txt"

columns = [
    "duration", "protocol_type", "service", "flag", "src_bytes", "dst_bytes", "land",
    "wrong_fragment", "urgent", "hot", "num_failed_logins", "logged_in",
    "num_compromised", "root_shell", "su_attempted", "num_root", "num_file_creations",
    "num_shells", "num_access_files", "num_outbound_cmds", "is_host_login",
    "is_guest_login", "count", "srv_count", "error_rate", "srv_error_rate",
    "rerror_rate", "srv_rerror_rate", "same_srv_rate", "diff_srv_rate",
    "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
    "dst_host_same_srv_rate", "dst_host_diff_srv_rate",
    "dst_host_same_src_port_rate", "dst_host_srv_diff_host_rate",
    "dst_host_error_rate", "dst_host_srv_error_rate", "dst_host_rerror_rate",
    "dst_host_srv_rerror_rate", "label", "difficulty"
]

df = pd.read_csv(train_path, header=None, names=columns)
df.drop("difficulty", axis=1, inplace=True)

cat_cols = ["protocol_type", "service", "flag"]
encoders = {}

#Encode categorical fields
for c in cat_cols:
    le = LabelEncoder()
    df[c] = le.fit_transform(df[c])
    encoders[c] = le

#Encoding labels
label_encoder = LabelEncoder()
df["label"] = label_encoder.fit_transform(df["label"])

X_train = df.drop("label", axis=1)
y_train = df["label"]

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

model = RandomForestClassifier(n_estimators=150, max_depth=20, random_state=42)
model.fit(X_train_scaled, y_train)

print("Model Ready!")
```

Fig4: Training Model Code Snippet

B. Evaluation Code Implementation:

Scikit-learn has multiple metrics that help evaluate model performance. We used: ***accuracy_score*** for displaying overall accuracy, ***classification_report*** for displaying class-level performance through precision, recall and F1-score.

Confusion matrix is generated using ***confusion_matrix*** and it is visualized using ***seaborn***, this helps identify correct classifications as well as common misclassification patterns among different attack types.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

#Load KDD Data
train_path = "KDD/train[1].txt"
test_path = "KDD/test[1].txt"

columns = [
    "duration", "protocol_type", "service", "flag", "src_bytes", "dst_bytes",
    "land", "wrong_fragment", "urgent", "hot", "num_failed_logins", "logged_in",
    "num_compromised", "root_shell", "su_attempted", "num_root", "num_file_creations",
    "num_shells", "num_access_files", "num_outbound_cmds", "is_host_login",
    "is_guest_login", "count", "srv_count", "serror_rate", "srv_error_rate",
    "rerror_rate", "srv_rerror_rate", "same_srv_rate", "diff_srv_rate",
    "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
    "dst_host_same_srv_rate", "dst_host_diff_srv_rate",
    "dst_host_same_src_port_rate", "dst_host_srv_diff_host_rate",
    "dst_host_rerror_rate", "dst_host_srv_rerror_rate", "dst_host_error_rate",
    "dst_host_srv_error_rate", "label", "difficulty"
]

train_df = pd.read_csv(train_path, header=None, names=columns)
test_df = pd.read_csv(test_path, header=None, names=columns)

train_df.drop("difficulty", axis=1, inplace=True)
test_df.drop("difficulty", axis=1, inplace=True)

#Encode categorical features
cat_cols = ["protocol_type", "service", "flag"]
le = LabelEncoder()

for col in cat_cols:
    train_df[col] = le.fit_transform(train_df[col])
    test_df[col] = le.transform(test_df[col])

#Split features and labels
X_train = train_df.drop("label", axis=1)
y_train = train_df["label"]

X_test = test_df.drop("label", axis=1)
y_test = test_df["label"]

#Normalize
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

#Train Random Forest classifier
model = RandomForestClassifier(n_estimators=150, max_depth=20, random_state=42)
model.fit(X_train, y_train)

#Evaluate
y_pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, zero_division=0))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred, labels=model.classes_)
plt.figure(figsize=(12,10))
sns.heatmap(cm, annot=True, fmt='d',
            xticklabels=model.classes_,
            yticklabels=model.classes_,
            cmap='Blues')
```

Fig5:Evaluating Model Code Snippet

C. GUI Code Components:

The ***Tkinter*** library is the main library needed for implementation of our graphical user interface. The function ***upload_csv()*** allows users to upload a CSV file that contains network traffic data. The uploaded data is then validated to ensure it matches the expected feature structure before preprocessing and classification.

The ***transform_val()*** function ensures that the categorical values in the uploaded file are correctly encoded. Then prediction results are shown in a text panel inside the GUI. The ***save_results()*** function enables users to store the output that includes classified values back into a CSV file which makes this system practical for real-world use.

```
#Button theme
style = ttk.Style()
style.theme_use("clam")

style.configure(
    "Pretty.TButton",
    font=("${Segoe UI}", 12, "bold"),
    padding=10,
    foreground="#FFFFFF",
    background="#5A338C",
    bordercolor="#5A338C",
    borderthickness=3,
    focusthickness=3,
    focuscolor="#A18CFF"
)

style.map(
    "Pretty.TButton",
    foreground=[('active', "#FFFFFF")],
    background=[('active', "#7154F9"), ('pressed', "#512E62")],
)

#Buttons
btn_frame = tk.Frame(root, bg="#5B8C88")
btn_frame.pack(pady=20)

upload_btn = ttk.Button(btn_frame, text="📁 Upload CSV & Classify", style="Pretty.TButton", command=upload_csv)
upload_btn.grid(row=0, column=0, padx=10)

save_btn = ttk.Button(btn_frame, text="💾 Save Results", style="Pretty.TButton", command=save_results)
save_btn.grid(row=0, column=1, padx=10)

#Results Panel
panel = tk.Frame(root, bg="#FFFFFF")
panel.pack(padx=30, pady=20, fill="both", expand=True)

panel_title = tk.Label(
    panel,
    text="📄 Classification Output",
    bg="#FFFFFF",
    fg="#473D63",
    font=("${Century Gothic}", 16, "bold")
)
panel_title.pack(pady=10)

results_box = tk.Text(
    panel,
    height=20,
    width=80,
    font=("${Consolas}", 12),
    bg="#F4F1FF",
    fg="#2A134F",
    bd=0,
    wrap="word"
)
results_box.pack(padx=20, pady=10)
results_box.config(state="disabled")

root.mainloop()
```

Fig4:GUI Code Snippet

7. Task Assignment

Student Name	Tasks Performed	Contribution (%)	Signature
Abdelrahman Wagdy	Analyzed research of free topic and prepared presentation slides	20%	Abdelrahman
Gamila Hasan	Developed the main Random Forest model code and contributed to the report	20%	Gamila
Mariam Reyad	Assisted in data preprocessing and evaluation and contributed to the report	20%	Mariam
Marina Ayman	Led the development of the GUI and data preprocessing and assisted in testing	20%	Marina
Mohamed Amr	Coordinated the research of the free topic and contributed to the report	20%	Mohamed